

HyGen: Generative Map Creation for Hytale

Brian Bowers
Department of Computer Science
Loyola Marymount University
Los Angeles, USA
bbowers3@lion.lmu.edu

August Wetterau
Department of Computer Science
Loyola Marymount University
Los Angeles, USA
awettera@lion.lmu.edu

Thomas Rife
Department of Computer Science
Loyola Marymount University
Los Angeles, USA
trife@lion.lmu.edu

Abstract—Map generation, often referred to as procedural generation, is crucial for modern game development. While some maps are fixed, other maps are uniquely generated, typically used in open-world games that have a large explorable area. *Hytale* is a fantasy sandbox role-playing game (RPG) that has gained popularity recently, in part due to its built-in tools for animation, modeling, and scripting that allow for accessible modding. Following the application track, we developed a learned generative model that generates small 256x256 block map layouts for the game *Hytale*.

I. PROBLEM STATEMENT AND MOTIVATION

Video games with large, explorable maps typically utilize procedural generation. Instead of having each map hand designed by humans, an algorithm is used. These algorithms use noise functions to generate unique terrain composed of similar features. Procedural generation is used in the game *No Man's Sky* to generate 18 quintillion planets, ensuring players never need to stop exploring. *Minecraft* uses procedural generation combined with a random seed number to generate virtually endless worlds, which can be recreated by any player with the seed. However, the limitation with traditional procedural generation approaches is that often players have no input on what terrain is created.

Hytale, a block based game similar to *Minecraft*, attempts to let designers have more control over the procedural generation process. Their *World Gen V2* system allows specific patterns to be implemented by players and designers, which can be used to ensure, for example, that glowing mushrooms only appear in sheltered areas [?]. While these rules can be an effective way to allow players to create custom mods and maps, it still requires users to hand craft these patterns. This is not only time consuming, but also knowledge intensive.

A popular feature of *Hytale*, and other similar games, is to allow players to make modifications to the basic game, called "mods". A few popular *Hytale* mods include boss fights, player-vs-player (PvP) arenas, and dungeon escape quests [?]. While many of these mods feature hand-crafted design, we envision users being able to simply describe the type of terrain they want in their world, and it will automatically be generated for them.

We decided to address this problem by created a learned generative model that can generate maps following users natural language instructions. With our project, players can quickly build any world that they can imagine.

II. SYSTEM DESIGN AND ARCHITECTURE

A. Project Overview

Our first approach was based off the intuition that map building, using height-maps, is very similar to image generation. We first built several traditional image generation models, including a variational auto-encoder (VAE) and a diffusion model. We trained these models of a procedurally generated dataset. While this produced decent results, we still wanted to create more playable and cohesive maps. To achieve this, we switched to a dataset of scraped maps, which contained more realistic terrain. Finally, we created a hybrid model that outperformed the traditional models. Finally, we implemented our model as a mod in *Hytale*, allowing users to create their own maps.

B. First Approach

We were unable to find any available datasets for *Hytale*, likely due to it being a new game. Therefore, we had to crete our own dataset. To do this, we used rule-based generation to create maps from seven different archetype. Each of these archetypes used hand crafted rules that we wrote, which led to them being composed of simple designs. Each map involved some random variation, but maps within the same archetype were quite similar. Specifially, the dataset was composed of heightmaps, meaning that they were 2-dimensional arrays were each valued represented the height the upmost block.

Using this dataset, we were able to train a VAE and generate new heightmaps by sampling from the latent space. The results from the VAE were quite blurry, as most of the sharp lines and defining features had been smoothed out. In addition, almost all the maps featured a circular pattern in the middle. This is likely due to the fact that we may have introduced an unintended bias into the dataset, as five of the seven archetypes featured circular shapes in the middle.

We then built a diffusion model, hoping it could address some of the limitations of the VAE. We found that the diffusion model generated much sharper images. In addition, it was able to faithfully capture the less represented archetypes present in our dataset. More importantly though, diffusion models can be conditioned on text embeddings, through cross-attention mechanisms, which was crucial for allowing players to control the terrain generation.

Additional details of our first approach can be found in our progress report.

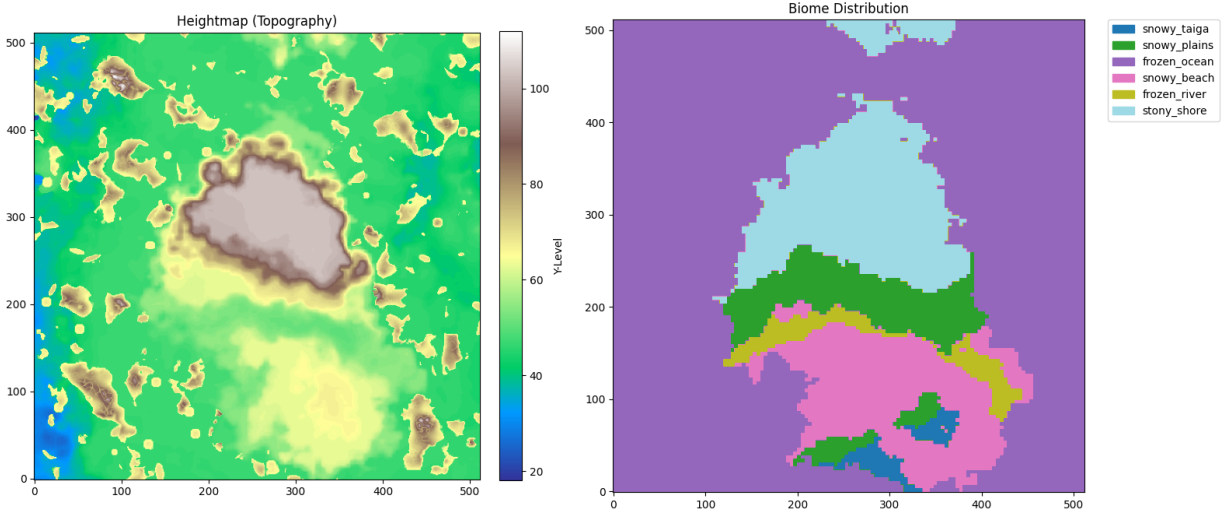


Fig. 1. Example from our final dataset.

C. Second Approach

When we implemented some of the generated maps from the first approach in game, we saw that they contained unrealistic features that would lead to the game being unplayable. These included lots of noise in parts that should be flat, terrain that was incredibly steep, and cut off features. We hypothesized that part of the problem was due to our procedurally generated dataset. We implemented samples from our dataset in game, and found that while the heightmaps looked good, the actual in game maps were quite poor.

To address this, we decided to build a new dataset, one which actually was based on real terrain in game. Essentially, games such as *Minecraft* and *Hytale*, have much more advanced procedural generation techniques, which we could not compete with. Therefore, it made sense to use their much better procedures to create a dataset. We decided to scrape maps from *Minecraft* because it contains a player built mod called *Tectonic* [?]. This mod recreates terrain-shaping in *Minecraft*, using more advanced functional noise programming and hardcoded rules. We chose this mod because it has land consolidation (land is grouped together in similar biomes), elevation blending (reduces sharp edges and gains in elevation), and biome smoothing (biomes switch gradually from one another). In addition, it contains aesthetic updates such as coastal cliffs, rolling dunes, and mountain belts.

We then implemented a script to build us a dataset containing scraped map features. Firstly, it generated a random seed, which let us load new map in the *Tectonic* mod. Then we extracted all blocks within 100 chunk radius (in *Minecraft* one chunk is 16 blocks). From those blocks, we removed artifacts such as trees, grass, and structures. This left us with a bare terrain, that we then chunked up into several 256 by 256 blocks. These patches were then converted into a heightmap.

In addition, we also extracted the biome type and created a matching biome distribution map. In addition, we used the map metadata to create a text label, representing the natural language input users would give to generate that terrain. An example from our dataset is shown in Figure 1.

D. Hybrid Approach

While our automated data pipeline provided an unlimited amounts of training data, our main design constraint was the training time. Additionally, diffusion models are very slow due to the gradual denoising process, which we experienced firsthand with our diffusion model from our first approach.

Our main design choice was to use a "warm start", meaning that instead of the diffusion model having to generate images from completely random noise, it would be given a starting point. To accomplish, we generated a macro world map, derived from the user prompt. By giving a global representation the model learns to tune the more-fine grained details, instead of having to learn to generate from scratch.

Furthermore, we used a VAE to compress the map before it is passed into the main diffusion model. This reduced the training time of the diffusion model significantly, in part because it reduces the dimensionality of the diffusion model. In addition, creating a latent representation can help the model by eliminating redundant information.

We decided to use a Conditional Diffusion U-Net as our main diffusion model, because for terrain generation, large-scale structure and fine-grained details must both be maintained. U-Nets excel at this, due to their encoder-decoder structure and skip connections. Furthermore, the U-Net can be conditioned on the text-embeddings and biome information, to generate terrain that aligns with user input.

Finally, we realized that the U-Net was outputting terrain chunks whose borders did not match, which led to visible seams between neighboring chunks. To address this issue, we added a ControlNet, which provides spatial conditioning based on the borders of the macro world map. This helps the model to generate terrain that complements its surroundings.

The architecture for the first part of our model, the Macro Terrain Generator, is shown in Figure 2.

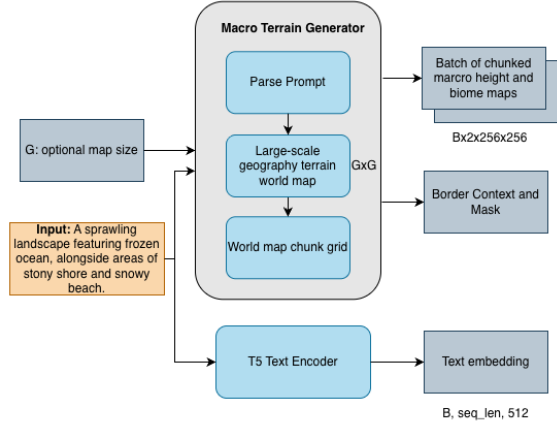


Fig. 2. Macro Terrain Generator.

The input into our system is natural language prompt provided by the user. The prompt is fed to our Macro Terrain Generator, which uses a semantic parser to identify the global terrain archetype. From this classification, a macro low-resolution world map is created. This map contains global features such as mountains belts and canyons, and is used as the starting point for our diffusion process. Additionally, the user can specify the size of the final output map. This means that the generated Macro Terrain world map size is variable. To address this, we chunk up the world map, resulting in a batch of 256 by 256 patches. We use a sliding-window strategy when chunking up the large map, as CNNs perform better with a overlapping windows. We also extract border context, containing additional information about the terrain features of adjacent chunks. Finally, we also have generate a border mask, which is used later when we are dealing with the seams in post-processing. Additionally, the user prompt is passed to a T5 Text Encoder, which generates a text embedding.

The batched 256 by 256 height maps, the border context, and directly used in the next steps of the model, shown in Figure 3.

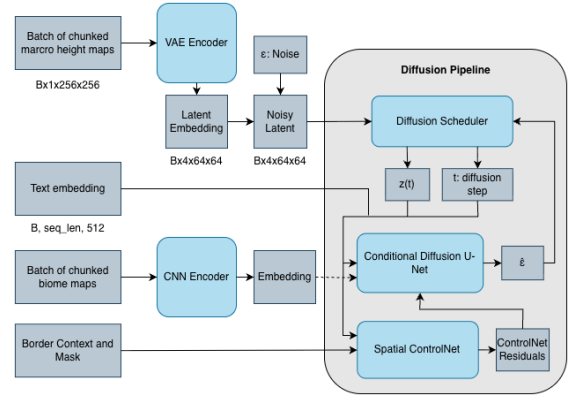


Fig. 3. Diffusion Pipeline.

Firstly, we pass the batched height maps to a VAE, which compresses the 256 by 256 patches into a smaller 64 by 64 latent dimension. Then, random noise (ϵ) is added to the latent embedding, resulting in the noisy latent. This is the direct input into our diffusion process, where the added noise will gradually be refined. In addition, the border context and the user prompt embedding are passed into the diffusion pipeline.

The diffusion pipeline is composed of three systems.

1) *Diffusion Scheduler*: This part is responsible for controlling the diffusion timestep, t , and handling the latent state. At each t , the diffusion scheduler passes the latent state and the timestep to both the U-Net and the ControlNet. Having a shared timestep is crucial for the synchronization of the two diffusion models. The scheduler then takes the output of the U-Net, which is predicted noise $\hat{\epsilon}$, and subtracts the noise from the previous latent state. This process is continued until the maximum number of steps is reached, and the final latent state is the final output from the diffusion pipeline.

2) *Conditional Diffusion U-Net*: The input to the U-Net is the latent state from the scheduler, the diffusion step t , the text embedding from the user prompt, and the residuals from the ControlNet. A U-Net is ideal because it is a specialized convolutional neural network, and our terrain heightmaps can be through of as patches of single channel (black and white) images. The encoder in the U-Net downsamples each noisy patch using convolutional layers, which enables the model to learn large-scale features. The decoder in the U-Net then up samples, reconstruction specific spatial details. In addition, skip connections help to ensure that the model does not forget details from the start of the process. Conditioning on the text embedding ensures over time the noise is shaped to align with the user prompt. In addition, the U-Net can optionally receive a biome embedding, which is created from using a CNN encoder on the biome map. This biome embedding additionally helps the model by guiding the terrain generation, as it provides clear separation between mountains, rivers, etc.

3) *Spatial ControlNet*: The input to the ControlNet is the shared diffusion timestep and latent state. The main job of the ControlNet it to output residuals, which tell the U-Net information about what features to add so that the chunks

match their neighbors. To do this, it must be at the same time step as the U-Net, otherwise it would be providing information about past states that may not be applicable to the current state. To help the ControlNet output useful residuals, it is conditioned on the user prompt text embedding, the border context, and the border mask. The border context and mask provides terrain boundaries of the surrounding regions. This helps the U-Net to ensure that the generated chunks match and there are not visible seams between bording chunks, allowing the map to be more geographically consistent and playable.

To enhance the diffusion models adherence to the user prompt we use Classifier-Free Guidance (CFG). This means that during training we would occasionally remove the T5 prompt. This forces the diffusion model to learn two noise predictions, which amplifies the user prompt. In addition, we used Exponential Moving Average (EMA) when updating the model weights. This helped to stabilize the training process and helps to ensure the generated terrain is natural. Finally, for the scheduler we uses

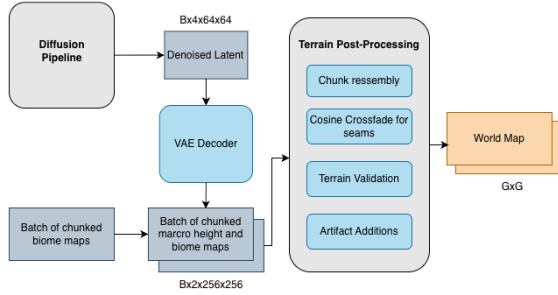


Fig. 4. Terrain Post-Processing.

The denoised latent is the final output of the diffusion pipeline. This is passed through the VAE decoder, which results in the final heightmap chunks of 256 by 256. We then add back in the biome chunks to the heightmaps, which becomes the input to our Terrain Post-Processing module. The main goal of this module is to resemble the chunks to recreate the world map. Each chunk has an overlapping window, which due to the Control-Net should have matching terrain. However, since there are two different generated terrain values for the borders, we need to decide which ones to use. To do this, we apply a cosine crossfade, to ensure we are using parts from both boarding chunks. Doing this helps to ensure that the chunks look natural, and that there are not any visible grid patterns in the world map from where the chunks were seperated. Furthermore, to use a validation step to ensure that there are not issues in the final world map such as floating terrain, impossible cliffs, and biome inconsistencies. This is the final terrain map that is output from the world map. Optionally, we also add in some artifacts, or terrain decorations, such as tress, plants, cacti, flowers, snow, etc. We

allowed users to choose whether they wanted these artifacts as often users will want to implement these themselves, and our focus is just on generating the actual terrain.

III. GENERATIVE AI INTEGRATION DETAILS

Our project focuses on solving an inherently generative process, creating terrain maps. While traditional procedural generation methods are deterministic and hand-crafted, we applied more advanced generative models to the problem. After exploring several different approaches, we settled on a hybrid method that procedurally generates a starting point, which is then compressed by a VAE, and input into our diffusion model. We used a diffusion model as our main source for creating maps, since it let us condition the outputs on the user input. This allows users to have more control over the generated terrain than traditional procedural generative approaches.

IV. EVALUATION AND USER FEEDBACK

V. AI ETHICS REFLECTION

VI. CONCLUSION

We built a learned generative AI model that generated world maps for the game *Hytale*.

Built a server and a mod so that users can use it in Game.

VII. FUTURE WORK

The prompt is fed to our Macro Terrain Generator, which starts by parsing the prompt looking for keywords, such as river, ocean, canyon, cliff, etc. Instead of parsing should classifier to group into different categories.